

Perfect  — here is **DAY 2 (FINAL, CORRECT, aligned with your working pdfplumber flow + template)**

👉 Focus = **ONLY TEXT EXTRACTION (PyMuPDF)**

👉 No tables yet (Day 3)

  **DAY 2 — TEXT EXTRACTION (PyMuPDF)**

  **WHERE WE ARE**

Day 1  Infra ready

Day 2 ← YOU ARE HERE

Tables  (next)

Schema 

  **GOAL**

PDF → clean text + titles 

  **FILES TO MODIFY**

app/parser/

└─ parser_router.py  modify

└─ pymupdf_parser.py  create

  **STEP 1 — INSTALL (if not done)**

pip install pymupdf

  **STEP 2 — CREATE TEXT PARSER**

 **app/parser/pymupdf_parser.py**

```
import fitz # PyMuPDF
```

```
def parse_pymupdf(file_path):
```

```
doc = fitz.open(file_path)
```

```
titles = []
```

```
text = []
```

```
for page_num, page in enumerate(doc):
```

```
    content = page.get_text()
```

```
    if not content:
```

```
        continue
```

```
    lines = content.split("\n")
```

```
    for line in lines:
```

```
        line = line.strip()
```

```
        if not line:
```

```
            continue
```

```
        #  Title detection (important)
```

```
        if (
```

```
            line.isupper() or
```

```
            "TEST REPORT" in line or
```

```
            "BRAKE" in line
```

```
        ):
```

```
            titles.append(line)
```

```
        else:
```

```
            text.append(line)
```

```
return {  
    "titles": titles,  
    "text": text  
}
```

✅✅ STEP 3 — UPDATE ROUTER

 **app/parser/parser_router.py**

```
from app.config import MODE  
  
from app.parser.pymupdf_parser import parse_pymupdf  
from app.parser.gcp_parser import parse_gcp  
  
def parse_document(file_path):  
    if MODE == "paid":  
        return parse_gcp(file_path)  
  
    # ✅✅ FREE MODE  
    return parse_pymupdf(file_path)
```

✅✅ STEP 4 — PIPELINE (NO CHANGE)

👉 already correct:

```
parsed = parse_document(file_path)
```

✅✅ STEP 5 — TEST

 **tests/test_ingestion.py**

```
from app.ingestion.pipeline import process_pdf  
  
result = process_pdf("data/sample.pdf")
```

```
print("\n=== TITLES ===\n", result["titles"][:10])
```

```
print("\n=== TEXT ===\n", result["text"][:20])
```

✅✅ EXPECTED OUTPUT

✅ TITLES

TPDS TEST REPORT ✅

BRAKE THERMAL MAPPING ✅

VEHICLE CONFIGURATION ✅

✅ TEXT

Objective: To evaluate brake thermal mapping ✅

Booster Size 9" ✅

Front Brake Pad Grade Apache ✅

✅✅ VERIFY (VERY IMPORTANT)

✅ You should see:

✅ 100+ text lines (not empty)

✅ meaningful engineering text

✅ titles separated

❌ If you see:

[] empty ❌

👉 then PyMuPDF not working

✅✅ FINAL STATE AFTER DAY 2

Text ✅

Titles 

Parser  (stable)

Tables  (next day)

WHY THIS STEP MATTERS

 This text will later give:

✓ metadata (component, dvp)

✓ fallback when tables fail

✓ context for LLM

NEXT

DAY 3 (IMPORTANT)

pdfplumber tables + None cleaning (your breakthrough)

 Say:

Day 3

We'll plug your working table extraction **properly into system** 